ASSIGNMENT NO. 8

Problem Statement: Implement Prim's algorithm to find the Minimum Spanning Tree (MST) cost of a weighted undirected graph.

Aim: To implement Prim's algorithm using a priority queue.

Objective:

1. To construct a Minimum Spanning Tree from a connected weighted graph.
2. To compute the minimum total cost required to connect all vertices.

Theory / Background: Prim's algorithm is a greedy approach for MST construction. It starts from any vertex and repeatedly chooses the minimum-weight edge that connects a visited vertex to an unvisited vertex. Using a min-priority queue improves efficiency for sparse graphs.

Algorithm Steps:

1. Initialize all vertices as unvisited.
2. Start from source vertex and push `(0, source)` in min-priority queue.
3. Pop the edge with minimum weight.
4. If destination vertex is unvisited, include its weight in MST and mark visited.
5. Push all edges from this vertex to unvisited neighbors.
6. Repeat until all vertices are visited.

Code:

```
#include <bits/stdc++.h>
using namespace std;

#define pii pair<int, int>

int prims(int V, vector<vector<pii>> &adj) {
    priority_queue<pii, vector<pii>, greater<pii>> pq
```

```
    vector<bool> visited(V, false);

    pq.push({0, 0});
    int totalCost = 0;

    while (!pq.empty()) {
        auto top = pq.top();
        pq.pop();

        int weight = top.first;
        int u = top.second;

        if (visited[u]) continue;

        visited[u] = true;
        totalCost += weight;

        for (auto &edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;

            if (!visited[v]) {
                pq.push({w, v});
            }
        }
    }

    return totalCost;
}

int main() {
    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    cout << "Enter number of edges: ";
    cin >> E;

    vector<vector<pii>> adj(V);

    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < E; i++) {
        int u, v, w;
        cin >> u >> v >> w;

        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    int result = prims(V, adj);
```

```
    cout << "Minimum Spanning Tree Cost: " << result

    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 7
Enter edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
Minimum Spanning Tree Cost: 16
```

Analysis:

- **Time Complexity:** O(E log V)
- **Space Complexity:** O(V + E)

Conclusion: Prim's algorithm efficiently computes the MST cost by always choosing the next minimum valid edge.

ASSIGNMENT NO. 9

Problem Statement: Implement a program to find Hamiltonian cycles in an undirected graph.

Aim: To detect and print all Hamiltonian cycles using backtracking.

Objective:

1. To understand Hamiltonian path/cycle constraints.
2. To implement recursive backtracking with safety checks.

Theory / Background: A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the start vertex. This is an NP-complete problem in general. Backtracking is used to build candidate paths and abandon invalid partial paths early.

Algorithm Steps:

1. Fix starting vertex as 0.
2. Try placing vertices one by one in the path.
3. Ensure selected vertex is adjacent to previous and not already used.
4. If path length reaches `V`, check whether last vertex connects to first.
5. If yes, print cycle; otherwise backtrack.

Code:

```
#include <bits/stdc++.h>
using namespace std;

int V;

bool isSafe(int v, vector<vector<int>> &graph, vector
    if (graph[path[pos - 1]][v] == 0)
        return false;
    for (int i = 0; i < pos; i++) {
        if (path[i] == v)
            return false;
    }
    return true;
}

void hamiltonianCycleUtil(vector<vector<int>> &graph,
```

```
    if (pos == V) {
        if (graph[path[pos - 1]][path[0]] == 1) {
            for (int i = 0; i < V; i++)
                cout << path[i] << " ";
            cout << path[0] << endl;
        }
        return;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, graph, path, pos)) {
            path[pos] = v;
            hamiltonianCycleUtil(graph, path, pos + 1
            path[pos] = -1;
        }
    }
}

void findHamiltonianCycles(vector<vector<int>> &graph
    vector<int> path(V, -1);
    path[0] = 0;
    hamiltonianCycleUtil(graph, path, 1);
}

int main() {
    int E;
    cout << "Enter number of vertices: ";
    cin >> V;

    cout << "Enter number of edges: ";
    cin >> E;

    vector<vector<int>> graph(V, vector<int>(V, 0));
    cout << "Enter edges (u v):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    cout << "Hamiltonian Cycles:\n";
    findHamiltonianCycles(graph);
    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 7
Enter edges (u v):
0 1
1 2
2 3
3 4
4 0
1 3
0 2
Hamiltonian Cycles:
0 1 2 3 4 0
0 2 1 3 4 0
0 4 3 1 2 0
0 4 3 2 1 0
```

Analysis:

- **Time Complexity:** O(V!) in worst case (backtracking).
- **Space Complexity:** O(V) for recursion and path storage.

Conclusion: Backtracking effectively explores and prints valid Hamiltonian cycles while pruning invalid partial paths.

ASSIGNMENT NO. 10

Problem Statement: Implement Dijkstra's algorithm for finding shortest path from a source vertex.

Aim: To compute single-source shortest paths in a weighted graph with non-negative edge weights.

Objective:

1. To apply greedy strategy using a min-priority queue.
2. To compute minimum distances from source to all vertices.

Theory / Background: Dijkstra's algorithm repeatedly picks the unprocessed vertex with minimum tentative distance and relaxes its outgoing edges. With a priority queue, it works efficiently on sparse graphs.

Algorithm Steps:

1. Initialize all distances as infinity and source as 0.
2. Push source in min-priority queue.
3. Pop closest vertex and relax all adjacent edges.
4. If shorter path to a neighbor is found, update and push neighbor.
5. Continue until queue becomes empty.

Code:

```
#include <bits/stdc++.h>
using namespace std;

void dijkstra(int V, vector<vector<pair<int, int>>> &
    priority_queue<pair<int, int>, vector<pair<int, i
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        pair<int, int> top = pq.top();
        pq.pop();
        int u = top.second;

        for (auto &edge : adj[u]) {
            int v = edge.first;
```

```
            int weight = edge.second;
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }

    cout << "Vertex\tDistance from Source\n";
    for (int i = 0; i < V; i++) {
        cout << i << "\t" << dist[i] << endl;
    }
}

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;

    vector<vector<pair<int, int>>> adj(V);
    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < E; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    int source;
    cout << "Enter source vertex: ";
    cin >> source;
    dijkstra(V, adj, source);
    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (u v weight):
0 1 2
0 2 4
1 2 1
1 3 7
2 4 3
3 4 1
Enter source vertex: 0
```

```
Vertex  Distance from Source
0       0
1       2
2       3
3       7
4       6
```

Analysis:

- **Time Complexity:** O((V + E) log V) with priority queue.
- **Space Complexity:** O(V + E)

Conclusion: Dijkstra's algorithm provides efficient shortest-path computation from a source in non-negative weighted graphs.

ASSIGNMENT NO. 11

Problem Statement: Implement Min Heap and Max Heap with insertion, deletion of root, and display operations.

Aim: To build a heap-based priority structure supporting dynamic operations.

Objective:

1. To implement heap insertion using upward heapify.
2. To implement root deletion using downward heapify.
3. To support both Min Heap and Max Heap behavior.

Theory / Background: A heap is a complete binary tree stored as an array. In Min Heap, parent <= children; in Max Heap, parent >= children. Insertions and deletions maintain heap property using local swaps, providing logarithmic updates.

Algorithm Steps:

1. Insert value at array end and heapify upward.
2. For delete root, replace root with last element.
3. Remove last element and heapify downward from root.
4. Display array representation of heap.

Code:

```
#include <bits/stdc++.h>
using namespace std;

class Heap {
    vector<int> heap;
    bool isMinHeap;

public:
    Heap(bool type) { isMinHeap = type; }

    bool compare(int parent, int child) {
        if (isMinHeap)
            return parent > child;
        else
            return parent < child;
    }
```

```
    void insert(int val) {
        heap.push_back(val);
        int i = static_cast<int>(heap.size()) - 1;
        while (i > 0 && compare(heap[(i - 1) / 2], he
            swap(heap[i], heap[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
    }

    void heapifyDown(int i) {
        int target = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < static_cast<int>(heap.size()) && c
            target = left;
        if (right < static_cast<int>(heap.size()) &&
            target = right;

        if (target != i) {
            swap(heap[i], heap[target]);
            heapifyDown(target);
        }
    }

    void deleteRoot() {
        if (heap.empty()) {
            cout << "Heap is empty\n";
            return;
        }
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) heapifyDown(0);
    }

    void display() {
        for (int x : heap) cout << x << " ";
        cout << endl;
    }
};

int main() {
    int choice, type;
    cout << "Choose Heap Type:\n";
    cout << "1. Min Heap\n2. Max Heap\n";
    cin >> type;

    Heap h(type == 1);
    do {
        cout << "\n1. Insert\n2. Delete Root\n3. Disp
        cout << "Enter choice: ";
        cin >> choice;
```

```
        switch (choice) {
            case 1: {
                int val;
                cout << "Enter value: ";
                cin >> val;
                h.insert(val);
                break;
            }
            case 2:
                h.deleteRoot();
                break;
            case 3:
                h.display();
                break;
            case 4:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice\n";
        }
    } while (choice != 4);

    return 0;
}
```

Output:

```
Choose Heap Type:
1. Min Heap
2. Max Heap
1

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 1
Enter value: 4

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 1
Enter value: 3

1. Insert
2. Delete Root
```

```
3. Display
4. Exit
Enter choice: 1
Enter value: 5

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 1
Enter value: 6

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 3
3 4 5 6

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 2

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 3
4 6 5

1. Insert
2. Delete Root
3. Display
4. Exit
Enter choice: 4
Exiting...
```

Analysis:

- **Time Complexity:**
  - *Insertion:* O(log n)
  - *Delete Root:* O(log n)
  - *Display:* O(n)
- **Space Complexity:** O(n)

Conclusion: Heap operations were implemented successfully for both Min Heap and Max Heap using array-based complete binary tree

representation.

ASSIGNMENT NO. 12

Problem Statement: Implement a simple data visualization application to generate a Word Cloud from input text.

Aim: To generate and display a Word Cloud using Python libraries.

Objective:

1. To understand frequency-based text visualization.
2. To use `wordcloud` and `matplotlib` for graphical representation of text.

Theory / Background: A word cloud visualizes words where size indicates frequency in the input text. Words appearing more often are displayed larger. It is useful for quick exploratory analysis of textual data.

Algorithm Steps:

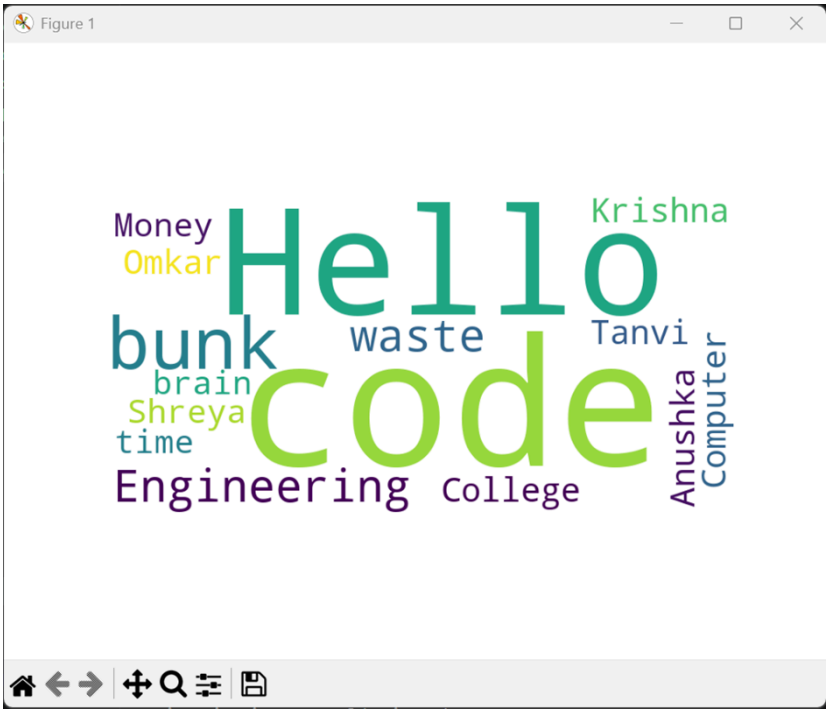
1. Accept text input from user.
2. Generate a Word Cloud object from text.
3. Render the generated cloud using matplotlib.
4. Hide plot axes and display the image.

Code:

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

text = input("Enter text:\n")
wc = WordCloud(width=800, height=400, background_colo
plt.imshow(wc)
plt.axis('off')
plt.show()
```

Output: Generated Word Cloud:



Analysis:

- **Time Complexity:** O(n), where `n` is length of input text for token counting.
- **Space Complexity:** O(k), where `k` is number of unique words.

Conclusion: The program successfully creates a frequency-based visual representation of input text using Python visualization libraries.